

SemiGuard AI - Database Code

데이터베이스 스키마 및 설정

생성일: Fri Aug 7 11:23:04 UTC 2026

파일: drizzle/schema.ts

```
import { float, int, mysqlEnum, mysqlTable, text, timestamp, varchar } from 'drizzle-orm';

export const users = mysqlTable("users", {
  id: int("id").autoincrement().primaryKey(),
  openId: varchar("openId", { length: 64 }).notNull().unique(),
  name: text("name"),
  email: varchar("email", { length: 320 }),
  loginMethod: varchar("loginMethod", { length: 64 }),
  role: mysqlEnum("role", ["user", "admin"]).default("user").notNull(),
  createdAt: timestamp("createdAt").defaultNow().notNull(),
  updatedAt: timestamp("updatedAt").defaultNow().onUpdateNow().notNull(),
  lastSignedIn: timestamp("lastSignedIn").defaultNow().notNull(),
});

export type User = typeof users.$inferSelect;
export type InsertUser = typeof users.$inferInsert;

// 이상 이력 로그 테이블
export const anomalyLogs = mysqlTable("anomaly_logs", {
  id: int("id").autoincrement().primaryKey(),
  userId: int("user_id").notNull(), // 사용자별 데이터 격리
  timestamp: timestamp("timestamp").defaultNow().notNull(),
  current: float("current").notNull(),
  temperature: float("temperature").notNull(),
  vibration: float("vibration").notNull(),
  noise: float("noise").notNull(),
  anomalyScore: float("anomaly_score").notNull(),
  riskLevel: mysqlEnum("risk_level", ["normal", "caution", "warning", "danger"]).default("normal").notNull(),
  isAnomaly: int("is_anomaly").notNull().default(0),
  llmAnalysisKo: text("llm_analysis_ko"),
  llmAnalysisEn: text("llm_analysis_en"),
  llmAnalysisJa: text("llm_analysis_ja"),
});

export type AnomalyLog = typeof anomalyLogs.$inferSelect;
export type InsertAnomalyLog = typeof anomalyLogs.$inferInsert;

// 데이터 공유 권한 테이블
export const dataSharing = mysqlTable("data_sharing", {
  id: int("id").autoincrement().primaryKey(),
  ownerId: int("owner_id").notNull(), // 데이터 소유자
```

```

    sharedWithUserId: int("shared_with_user_id").notNull(), // 공유 대상 사용자
    permission: mysqlEnum("permission", ["view", "edit"]).default("view").notNull(),
    createdAt: timestamp("createdAt").defaultNow().notNull(),
  });

export type DataSharing = typeof dataSharing.$inferSelect;
export type InsertDataSharing = typeof dataSharing.$inferInsert;

// 방문자 카운터 테이블
export const visitorStats = mysqlTable("visitor_stats", {
  id: int("id").autoincrement().primaryKey(),
  date: varchar("date", { length: 10 }).notNull().unique(), // YYYY-MM-DD
  count: int("count").notNull().default(0),
  updatedAt: timestamp("updatedAt").defaultNow().onUpdateNow().notNull(),
});

export type VisitorStat = typeof visitorStats.$inferSelect;

// 전체 샘플 카운터 (정상 포함 모든 측정값 집계용)
export const sampleStats = mysqlTable("sample_stats", {
  id: int("id").autoincrement().primaryKey(),
  key: varchar("key", { length: 32 }).notNull().unique(), // 'total_sample'
  value: int("value").notNull().default(0),
  updatedAt: timestamp("updatedAt").defaultNow().onUpdateNow().notNull(),
});

export type SampleStat = typeof sampleStats.$inferSelect;

// 위험도 임계값 설정 테이블 (싱글톤 row: key='default')
export const thresholdSettings = mysqlTable("threshold_settings", {
  id: int("id").autoincrement().primaryKey(),
  key: varchar("key", { length: 32 }).notNull().unique(), // 'default'
  normalMax: int("normal_max").notNull().default(29),
  cautionMax: int("caution_max").notNull().default(49),
  warningMax: int("warning_max").notNull().default(69),
  updatedAt: timestamp("updatedAt").defaultNow().onUpdateNow().notNull(),
});

export type ThresholdSetting = typeof thresholdSettings.$inferSelect;

// 센서별 임계값 설정 테이블 (싱글톤 row: key='default')
export const sensorThresholds = mysqlTable("sensor_thresholds", {
  id: int("id").autoincrement().primaryKey(),
  key: varchar("key", { length: 32 }).notNull().unique(), // 'default'
});

```

```

// 전류 (A) 임계값
currentCaution: float("current_caution").notNull().default(7.0),
currentWarning: float("current_warning").notNull().default(9.0),
currentDanger: float("current_danger").notNull().default(11.0),
// 온도 (°C) 임계값
tempCaution: float("temp_caution").notNull().default(55.0),
tempWarning: float("temp_warning").notNull().default(70.0),
tempDanger: float("temp_danger").notNull().default(85.0),
// 진동 (g) 임계값
vibCaution: float("vib_caution").notNull().default(0.6),
vibWarning: float("vib_warning").notNull().default(0.8),
vibDanger: float("vib_danger").notNull().default(1.0),
// 소음 (dB) 임계값
noiseCaution: float("noise_caution").notNull().default(65.0),
noiseWarning: float("noise_warning").notNull().default(75.0),
noiseDanger: float("noise_danger").notNull().default(85.0),
updatedAt: timestamp("updated_at").defaultNow().onUpdateNow().notNull(),
});

export type SensorThreshold = typeof sensorThresholds.$inferSelect;

// 사용자별 통계 테이블 (각 사용자의 절감 비용, 이상 탐지 횟수 등)
export const userStats = mysqlTable("user_stats", {
  id: int("id").autoincrement().primaryKey(),
  userId: int("user_id").notNull().unique(),
  totalAnomalies: int("total_anomalies").notNull().default(0),
  totalSavedCost: int("total_saved_cost").notNull().default(0),
  dangerResetOffset: int("danger_reset_offset").notNull().default(0),
  updatedAt: timestamp("updated_at").defaultNow().onUpdateNow().notNull(),
});

export type UserStat = typeof userStats.$inferSelect;
export type InsertUserStat = typeof userStats.$inferInsert;

```

파일: drizzle/relations.ts

```
import {} from "./schema";
```

파일: drizzle.config.ts

```
import { defineConfig } from "drizzle-kit";

const connectionString = process.env.DATABASE_URL;
if (!connectionString) {
  throw new Error("DATABASE_URL is required to run drizzle commands");
}

export default defineConfig({
  schema: "./drizzle/schema.ts",
  out: "./drizzle",
  dialect: "mysql",
  dbCredentials: {
    url: connectionString,
  },
});
```

파일: shared/types.ts

```
/**
 * Unified type exports
 * Import shared types from this single entry point.
 */

export type * from "../drizzle/schema";
export * from "../_core/errors";
```